

→ Javascript

↳ Synchronous

↳ Single threaded

Default

→ Execution Context

↳ execute one line of code at a time

→ console log → 1

→ console log → 2

each operation waits for the last one to complete before executing

CALL Stack

Memory Heap

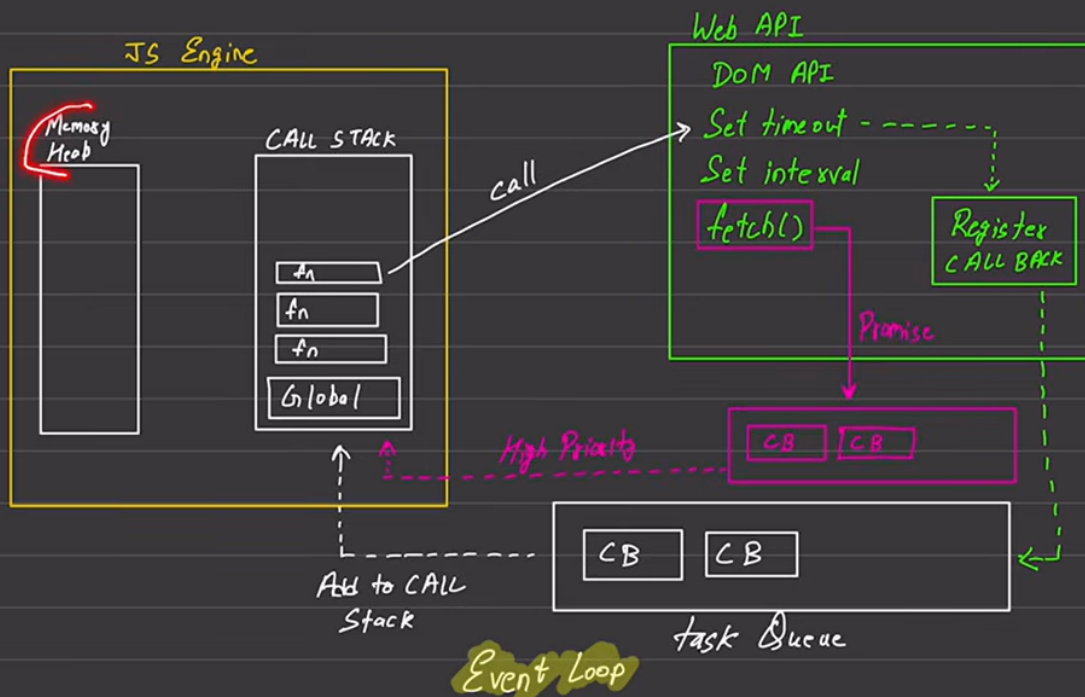
Blocking Code VS Non Blocking code

↓
Block the flow of Program

↓
Read File Sync

↳ Does not block execution

↓
Read File Async



Async JavaScript — Easy Explanation

Since you're learning JavaScript, understand **synchronous vs asynchronous** first.

1. Synchronous JavaScript

Normally, JavaScript executes code **line by line**.

```
console.log("1");  
console.log("2");  
console.log("3");
```

Output:

```
1  
2  
3
```

JavaScript waits for each line to finish before moving to the next one.

Think:

"First finish this, then go to the next task."

2. What is Asynchronous JavaScript?

Asynchronous JavaScript allows JavaScript to **start a time-consuming task and continue doing other work instead of waiting for it to finish**.

Example:

```
console.log("Start");  
  
setTimeout(() => {  
    console.log("Inside timeout");  
}, 2000);  
  
console.log("End");
```

Output:

```
Start
End
Inside timeout
```

Notice something interesting:

```
Start
End
Inside timeout
```

Even though the `setTimeout()` appears before "End", "End" executes first.

Why?

Because `setTimeout()` is an **asynchronous operation**.

3. Real-life example

Imagine you're at a restaurant.

Synchronous

You:

```
Order food
↓
Stand in kitchen
↓
Wait for food
↓
Eat
```

You can't do anything else while waiting.

Asynchronous

You:

```
Order food
↓
Sit at table
↓
Use your phone / talk to friends
```

```
↓  
Food is ready  
↓  
Eat
```

You don't block your other work while waiting.

That's the basic idea of **async JavaScript**.

4. Why do we need Async JavaScript?

Some operations take time:

- API requests
- Database requests
- File operations
- Timers
- Network requests
- Fetching data from a server

For example:

```
fetch("https://example.com/data");
```

The server might take some time to respond.

If JavaScript completely stopped while waiting, your webpage would feel frozen.

Instead, JavaScript can continue executing other code.

5. `setTimeout()` example

```
console.log("A");  
  
setTimeout(() => {  
  console.log("B");  
}, 2000);  
  
console.log("C");
```

Output:

A
C
B

What happens?

Initially:

```
console.log("A")  
  ↓  
  A  
  
setTimeout()  
  ↓  
Browser handles the timer  
  
console.log("C")  
  ↓  
  C
```

After 2 seconds:

B

6. Important: JavaScript is single-threaded

This is a very important concept.

JavaScript itself has **one main call stack**.

You can think of it as:

```
JavaScript  
  |  
Call Stack  
  |  
One task at a time
```

So how can JavaScript handle asynchronous tasks?

This is where the **Browser Web APIs + Callback Queue + Event Loop** come in.

7. Event Loop

This is one of the most important concepts in JavaScript.

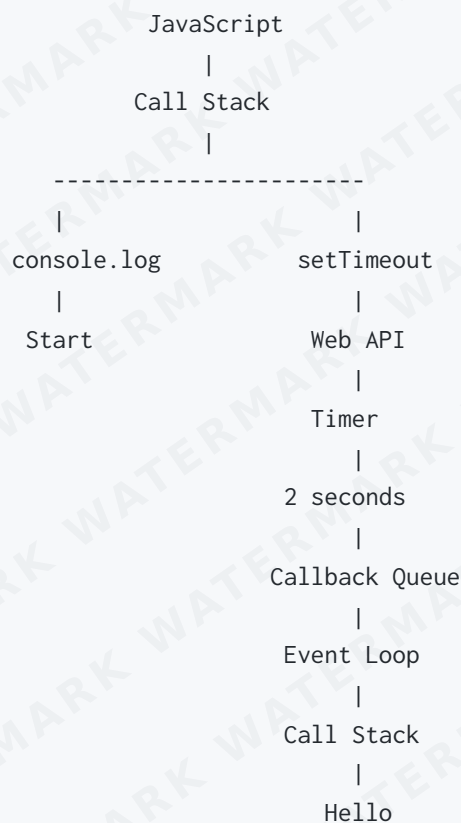
Suppose:

```
console.log("Start");

setTimeout(() => {
  console.log("Hello");
}, 2000);

console.log("End");
```

The simplified process is:



The **Event Loop** keeps checking:

"Is the Call Stack empty?"

If yes, it can move a callback from the queue to the Call Stack.

8. Callback

A callback is simply a **function passed to another function**.

Example:

```
function greet() {  
  console.log("Hello");  
}  
  
setTimeout(greet, 2000);
```

Here:

```
greet
```

is the callback function.

After 2 seconds, it gets executed.

9. Promises

Promises are another major part of asynchronous JavaScript.

A Promise represents a value that we **don't have yet but expect to get in the future**.

A Promise has three states:

```
Pending  
↓  
Resolved
```

or

```
Pending  
↓  
Rejected
```

Example:

```
const promise = new Promise((resolve, reject) => {
```

```
let success = true;

if (success) {
  resolve("Data received");
} else {
  reject("Something went wrong");
}

});
```

10. .then() and .catch()

We can handle the result of a Promise using:

```
promise
  .then((result) => {
    console.log(result);
  })
  .catch((error) => {
    console.log(error);
  });
```

then() → when Promise succeeds.

catch() → when Promise fails.

11. async and await

Modern JavaScript commonly uses:

```
async
await
```

Example:

```
async function getData() {

  const response = await fetch("https://example.com");

  console.log(response);
}
```

```
}
```

`async`

When you put `async` before a function:

```
async function getData() {  
  
}
```

the function automatically returns a **Promise**.

`await`

`await` means:

"Wait for this Promise to settle before continuing this async function."

Example:

```
async function getData() {  
  
    console.log("Start");  
  
    const response = await fetch("https://example.com");  
  
    console.log("Data received");  
  
    console.log("End");  
}
```

The important point is that `await` **doesn't freeze the entire JavaScript program**. It pauses that particular async function while other work can continue.

12. Error handling with `try...catch`

With `async/await`, we commonly use:

```
async function getData() {  
  
    try {
```

```
const response = await fetch("https://example.com");

console.log(response);

} catch (error) {

  console.log(error);

}

}
```

Think:

```
try
↓
Run async operation
↓
Success → continue
↓
Failure → catch
```

13. The complete picture

You should understand async JS in roughly this order:

```
Synchronous JS
↓
Asynchronous JS
↓
Web APIs
↓
Callbacks
↓
Callback Queue
↓
Event Loop
↓
Promises
↓
.then() / .catch()
↓
async / await
↓
try / catch
```

One-line definition for exams/interviews:

Asynchronous JavaScript allows time-consuming operations to run without blocking the execution of other JavaScript code, using mechanisms such as callbacks, promises, and `async/await`.

And remember the most important distinction:

Synchronous → Wait and then continue

Asynchronous → Start the task, continue other work,
handle the result when it is ready